COMPTE RENDU DE PROJET

DÉVELOPPEMENT WEB

MiiProject

Plateforme de mini-jeux multijoueur en ligne

Wierez Mylo

Groupe avec :

Théo, Alexis,

Marylou

B1 — 2025-26

Rendu le : 12/04/2026

Sommaire

Présentation
- Thème
- Objectifs

Analyse du sujet
- Contexte et besoins
- Fonctionnalités cibles

Méthodologie
- Approche
- Répartition du travail
- Outils et ressources

Réalisation
- Architecture technique
- Les mini-jeux
- Infrastructure d'hébergement
- Fonctionnalités avancées
- Sécurité et déploiement

Synthèse
- Bilan général
- Difficultés rencontrées
- Limites identifiées
- Améliorations possibles

Conclusion

Présentation

Thème

Ce projet consiste en la conception et le développement d'une plateforme web de mini-jeux multijoueur en ligne, nommée MiiProject, réalisée dans le cadre du cours de développement web. Son objectif est de mettre en pratique les notions suivantes :

- Développement frontend en React avec TypeScript
- Gestion de la communication temps réel via Supabase Realtime
- Déploiement d'une application conteneurisée sur infrastructure personnelle (Proxmox)
- Mise en place d'un reverse proxy et d'une configuration DNS sécurisée
- Internationalisation d'une interface web (FR/EN)

Objectifs

L'objectif fonctionnel est de produire une plateforme web permettant à plusieurs utilisateurs de rejoindre des salons de jeu en temps réel, de jouer à des mini-jeux compétitifs, et de disposer d'un système de comptes persistants avec personnalisation du profil.

L'objectif pédagogique est d'appréhender la complexité d'un projet full-stack complet : de la conception UI au déploiement en production, en passant par la gestion des états temps réel, la containerisation Docker et la sécurisation via Cloudflare.

Analyse du sujet

Contexte et besoins

L'idée de départ est simple : proposer une expérience de jeu en ligne légère, rapide et accessible, sans installation requise. L'utilisateur ouvre le site, saisit un pseudo ou se connecte à son compte, crée ou rejoint un salon via un code à 5 caractères, puis joue immédiatement avec ses amis.

Ce type de plateforme, popularisé par des jeux comme Jackbox Party Pack, repose sur plusieurs contraintes techniques précises : la synchronisation d'état en temps réel entre tous les joueurs, la gestion des déconnexions, et une interface réactive sur mobile comme sur desktop.

Fonctionnalités cibles

Les fonctionnalités identifiées dès l'analyse du besoin sont les suivantes :

- Système de salons multijoueur avec codes de partage uniques
- Trois mini-jeux complets : Pictionary, Réaction, Trivia
- Mode solo avec difficulté paramétrable pour Réaction et Trivia
- Système de comptes utilisateurs (inscription, connexion, profil personnalisable)
- Traduction intégrale FR/EN avec détection automatique
- Questions de trivia dynamiques (Open Trivia Database + traduction LibreTranslate)
- Déploiement auto-hébergé sur serveur Proxmox personnel

Méthodologie

Approche

Le développement a suivi une démarche itérative, s'appuyant entièrement sur la génération de code assistée par IA via la plateforme Lovable. Ce choix constitue lui-même un objectif du projet : démontrer qu'il est possible de produire une application web complexe et fonctionnelle sans écrire la moindre ligne de code manuellement.

Le workflow adopté est le suivant :

1. Rédaction d'un prompt de spécification détaillé envoyé à Lovable
2. Génération automatique du code React + TypeScript + Supabase
3. Synchronisation du dépôt GitHub généré vers le LXC Proxmox
4. Build et déploiement via Docker Compose
5. Tests fonctionnels, identification des bugs, corrections ciblées
6. Itérations successives pour ajout de fonctionnalités

Répartition du travail

Le projet a été réalisé en binôme. La conception des prompts, les tests fonctionnels et la configuration de l'infrastructure ont été partagés entre les deux membres. ANDRIES Alexis a pris en charge l'intégralité de la partie infrastructure : Proxmox, Docker, Nginx Proxy Manager, Cloudflare DNS, LibreTranslate et la configuration de sécurité (Fail2Ban, UFW).

Outils et ressources

Outil	Usage
Lovable	Génération de code IA (React/TS/Supabase)
Supabase	Backend BaaS : base de données, auth, Realtime
Docker / Docker Compose	Conteneurisation et déploiement
Proxmox VE 9.x	Hyperviseur hébergeant les LXC
Nginx Proxy Manager	Reverse proxy centralisé + SSL Let's Encrypt
Cloudflare	DNS, protection DDoS, cache, masquage IP
LibreTranslate	Traduction EN→FR auto-hébergée (port 5001)
GitHub (org MiiProject)	Versioning du code source
WebStorm	IDE connecté en SSH au LXC pour les éditions manuelles

Réalisation

Architecture technique

1. Stack frontend

L'application frontend est développée en React 18 avec TypeScript, stylisée via Tailwind CSS et animée avec Framer Motion. Le routing est géré par React Router, la communication temps réel par le client Supabase Realtime via des channels de broadcast.

La structure du projet suit une organisation par domaine fonctionnel :

- src/components/games/ — les 6 composants de jeu (multi et solo)
- src/components/ — composants UI partagés (lobby, scoreboard, auth…)
- src/hooks/ — logique métier externalisée (useRoom, usePlayerIdentity)
- src/i18n/ — fichiers de traduction FR et EN
- src/lib/ — types partagés et générateur de questions

2. Backend Supabase

Supabase joue le rôle de backend complet : base de données PostgreSQL pour les profils et statistiques, authentification email/password, et Realtime pour la synchronisation des états de jeu entre joueurs. Les channels Realtime sont utilisés en mode broadcast pour diffuser les états de jeu à tous les participants d'un salon.

Les tables principales de la base de données sont les suivantes :

Table	Contenu
profiles	Profil utilisateur : pseudo, avatar, bio, stats
game_stats	Historique des parties : type, score, résultat
rooms	Salons actifs : code, gameType, statut, gameState

3. Conteneurisation

L'application est conteneurisée via un Dockerfile multi-stage : un premier stage Node.js compile le frontend React, un second stage node:20-alpine exécute Nginx (fichiers statiques) et le serveur Express d'authentification sur le port 3001. Un script start.sh orchestre le démarrage des deux processus.

Le docker-compose.yml expose le port 3000, monte un volume persistant /data pour le fichier users.json, et injecte les variables d'environnement Supabase depuis le fichier .env.

Les mini-jeux

1. Pictionary

Le Pictionary est un jeu de dessin en temps réel. Un joueur tire un mot au sort et doit le dessiner sur un canvas HTML5 pendant que les autres tentent de le deviner via le chat intégré. Le dessin est synchronisé en temps réel via Supabase Realtime : chaque trait est broadcasté sous forme de coordonnées et reproduit chez tous les spectateurs.

Les points sont attribués en fonction de la rapidité de la réponse correcte : le système calcule un bonus décroissant basé sur le temps restant. Le dessinateur obtient des points proportionnellement au nombre de joueurs ayant deviné correctement.

2. Réaction

Le jeu de Réaction teste les réflexes des joueurs. Une forme colorée apparaît à un emplacement aléatoire de l'écran après un délai variable (entre 1 et 4 secondes). Le premier joueur à cliquer ou taper l'écran remporte le round et marque 100 points. Plusieurs rounds sont joués successivement, avec un tableau des scores mis à jour en direct après chaque round.

En mode solo, le joueur affronte un bot dont le temps de réaction est paramétré selon la difficulté choisie (Facile : 800ms, Moyen : 400ms, Difficile : 150ms).

3. Trivia

Le Trivia est un jeu de questions à choix multiples. Les questions sont récupérées dynamiquement depuis l'API Open Trivia Database (opentdb.com), garantissant une infinité de questions différentes à chaque partie. Tous les joueurs voient la même question simultanément et disposent de 15 secondes pour répondre.

Les points sont calculés selon la formule suivante :

```
points = max(50, timer_restant × 10)
```

Une fonctionnalité de traduction à la demande est intégrée : un bouton « 🌐 Traduire » apparaît sur chaque question et envoie la question ainsi que les quatre réponses à une instance LibreTranslate auto-hébergée, qui retourne la traduction en français en moins d'une seconde.

En mode solo, le joueur peut choisir le nombre de questions (5, 10 ou 25) ainsi que la difficulté. Il n'y a pas de bot adverse pour ce jeu : le score final reflète uniquement la performance personnelle du joueur.

Infrastructure d'hébergement

1. Architecture Proxmox

L'ensemble des services est hébergé sur un serveur Proxmox VE personnel. Chaque service est isolé dans un LXC (Linux Container) dédié, partageant le noyau hôte tout en restant entièrement isolés au niveau réseau et système de fichiers. Cette approche offre une densité bien supérieure à la virtualisation complète.

LXC ID	Nom	IP	Rôle
101	miiproject	172.23.200.220	Application MiiProject (Docker)
113	nginx-proxy	172.23.200.213	Nginx Proxy Manager (reverse proxy)
LibreTranslate	libretranslate	172.23.71.92	API de traduction auto-hébergée

2. Flux réseau

Le flux réseau suit le chemin suivant depuis un utilisateur externe :

```
Utilisateur → Cloudflare (cache + DDoS) → IP publique 82.65.164.120
→ UCG Fiber (port forwarding 80/443) → Nginx Proxy Manager
→ LXC miiproject:3000 → Docker → Nginx interne → React / Express
```

Le proxy Nginx interne au conteneur est également configuré pour router les requêtes /api/translate vers LibreTranslate en interne (172.23.71.92:5001), évitant ainsi d'exposer publiquement l'API de traduction.

Fonctionnalités avancées

1. Système d'authentification

Un système d'authentification complet a été développé en parallèle du mode invité existant. Il repose sur un serveur Express embarqué dans le conteneur, exposant quatre endpoints REST :

- POST /auth/register — inscription avec email, mot de passe (PBKDF2) et pseudo
- POST /auth/login — connexion avec vérification du hash
- PUT /auth/update/:id — modification du profil (pseudo, email, mot de passe)
- DELETE /auth/delete/:id — suppression du compte et de ses données

Les données sont stockées dans un fichier users.json persisté via un volume Docker, monté à /data à l'intérieur du conteneur. Cette approche a été choisie pour sa simplicité, le nombre d'utilisateurs attendus restant limité dans le cadre de ce projet.

L'interface utilisateur présente un bouton « Connexion / Inscription » lorsque l'utilisateur n'est pas connecté. Une fois authentifié, le pseudo s'affiche en haut à gauche et donne accès à la page de profil, depuis laquelle il est possible de modifier toutes les informations personnelles.

2. Internationalisation

L'intégralité de l'interface est traduite en français et en anglais via la librairie react-i18next. Les fichiers de traduction sont organisés par domaine fonctionnel (common, home, auth, games, room, scoreboard…) et couvrent tous les textes de l'application : labels, messages d'erreur, instructions de jeu, tooltips.

Un bouton de changement de langue est visible en permanence dans le header. La langue par défaut est le français, et la préférence est persistée dans le localStorage du navigateur entre les sessions.

3. Traduction dynamique des questions

Les questions de Trivia étant issues d'une API anglophone, un mécanisme de traduction à la demande a été implémenté. Chaque question affiche un bouton « Traduire » qui déclenche une requête vers l'instance LibreTranslate auto-hébergée. La traduction couvre simultanément la question et les quatre réponses via Promise.all, puis remplace le texte affiché sans interrompre le timer.

Sécurité et déploiement

1. Cloudflare

Le domaine andries.icu a été migré d'Ionos vers Cloudflare pour bénéficier de la protection DDoS, du cache CDN et du masquage de l'IP publique. Les nameservers Ionos ont été remplacés par arnold.ns.cloudflare.com et betty.ns.cloudflare.com. Tous les enregistrements A sont en mode Proxied (nuage orange), à l'exception du sous-domaine VPN.

Les options Bot Fight Mode et Block AI Bots sont activées pour filtrer le trafic automatisé malveillant. Le mode SSL/TLS est configuré en Full (strict), garantissant le chiffrement de bout en bout entre Cloudflare et le serveur d'origine.

2. Protection de LibreTranslate

L'API LibreTranslate est protégée par une clé API PBKDF2 générée via ltmanage. Les requêtes de traduction transitent par le proxy Nginx du conteneur MiiProject (/api/translate → 172.23.71.92:5001), de sorte que l'API n'est jamais exposée directement sur internet. Le sous-domaine translate.andries.icu est maintenu en DNS-only (désactivé) pour un usage de maintenance ponctuel uniquement.

3. Processus de déploiement

Le déploiement suit un workflow Git simple et reproductible :

7. Modification du code via Lovable (push automatique vers GitHub) ou directement sur le LXC via WebStorm en SSH
8. Sur le LXC : git pull pour récupérer les dernières modifications
9. Rebuild et redéploiement : docker compose up -d --build
10. Vérification des logs : docker compose logs -f

Synthèse

Bilan général

Le projet MiiProject remplit ses objectifs principaux : la plateforme est fonctionnelle, accessible en ligne depuis n'importe quel appareil, et propose trois mini-jeux jouables en multijoueur temps réel. L'ensemble des fonctionnalités planifiées a été livré : comptes utilisateurs, mode solo, traduction dynamique, déploiement containerisé et sécurisation via Cloudflare.

L'approche 100% IA pour la génération de code s'est révélée efficace pour atteindre un niveau de complexité technique élevé en un temps réduit. Elle a cependant nécessité un travail de compréhension, de débogage et de configuration infrastructure conséquent, démontrant que la maîtrise de l'environnement d'exécution reste indispensable même lorsque le code est généré automatiquement.

Difficultés rencontrées

- La synchronisation des états de jeu via Supabase Realtime : les broadcasts ne retournaient pas les messages à l'émetteur, causant des boucles infinies dans Pictionary et un affichage incorrect dans Réaction.
- La gestion des droits GitHub au sein de l'organisation MiiProject : le LXC de déploiement n'avait pas les permissions d'écriture, nécessitant une configuration SSH et une attribution de rôle.
- La politique CORS de l'API Anthropic bloquant les appels directs depuis le frontend, résolue par un proxy Nginx interne.
- L'incompatibilité de @discordjs/opus avec Node.js 20 lors d'un projet parallèle, résolue par le remplacement par opusscript.
- La configuration du nginx.conf avec node:20-alpine (Alpine installe Nginx sans conf.d), nécessitant de passer à /etc/nginx/nginx.conf avec le bloc http complet.

Limites identifiées

- Le stockage des comptes utilisateurs dans un fichier JSON n'est pas adapté à une montée en charge. Une base de données relationnelle (PostgreSQL via Supabase) serait préférable pour un déploiement à grande échelle.
- L'absence de validation côté serveur pour les actions de jeu : un joueur malveillant pourrait théoriquement manipuler les scores en forgant des messages Realtime.
- LibreTranslate, bien que fonctionnel, produit des traductions parfois approximatives pour les questions de culture générale. Les noms propres et les questions contextuelles sont mal rendus.
- Le mode solo de Pictionary repose sur un bot dont le comportement de dessin est simulatif et non réaliste, limitant l'intérêt pédagogique du mode.

Améliorations possibles

- Migration des données utilisateurs de users.json vers Supabase Auth pour une gestion native, sécurisée et scalable.
- Ajout d'une validation serveur des scores (hash de vérification côté Supabase Edge Functions).
- Implémentation d'un système de rooms persistantes avec historique des parties.
- Ajout de nouveaux mini-jeux : Loup-Garou, Blind Test musical, ou quiz de culture web.
- Mise en place d'un pipeline CI/CD via GitHub Actions pour déclencher automatiquement le rebuild Docker à chaque push sur main.
- Amélioration du bot Pictionary : utilisation d'un modèle de génération de descriptions textuelles pour rendre les indices plus pertinents.

Conclusion

MiiProject constitue une démonstration concrète et aboutie des compétences acquises en développement web, infrastructure et DevOps. En partant d'un simple prompt de spécification, le projet a produit une application web complète, déployée en production sur une infrastructure personnelle, accessible depuis n'importe où dans le monde sous le domaine miiproject.andries.icu.

Au-delà du code généré par l'IA, c'est la maîtrise de l'environnement d'exécution qui a constitué le véritable défi : comprendre Docker et ses volumes, configurer Nginx comme proxy, gérer la propagation DNS via Cloudflare, sécuriser LibreTranslate derrière un proxy interne, ou encore diagnostiquer des bugs de synchronisation Supabase Realtime. Ces compétences d'infrastructure sont indissociables du développement web moderne.

L'approche de génération de code par IA, bien qu'encore perfectible, ouvre une nouvelle façon de concevoir le développement logiciel : le rôle du développeur évolue vers celui d'architecte et d'intégrateur, capable de spécifier précisément un besoin, d'évaluer le code produit, et de le déployer dans un environnement maîtrisé. Ce projet aura été une illustration directe de ce paradigme.

La méthode adoptée — analyse du besoin, rédaction de prompts structurés, déploiement itératif, tests et corrections — s'est révélée reproductible et efficace. Elle constituera un gabarit pour les prochains projets de développement.